

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

GuardPath

PONDERACIÓN: 24

Horas Aproximadas: 60

Contenido

1. Resumen Ejecutivo	3
2. Competencias que desarrollaremos	3
3. Objetivos del Aprendizaje.....	4
3.1 Objetivo General.....	4
3.2 Objetivos Específicos	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema a resolver.....	4
4.2 Alcance del proyecto	5
4.3 Requerimientos técnicos	7
4.4 Entregables	8
5. Metodología	9
6. Desarrollo de Habilidades Blandas	11
6.1 Proyectos en Grupo.....	11
6.2 Proyectos Individuales.....	12
7. Cronograma.....	13
8. Rúbrica de Calificación	13
8.1 Requisitos para optar a la calificación.....	13
8.2 Resumen de Puntuaciones.....	14
8.3 Detalle de la Calificación	14
8.4 Valores	16

1. Resumen Ejecutivo

PathGuard es un simulador tridimensional de rescate autónomo desarrollado por estudiantes del curso de Inteligencia Artificial 1 de la Universidad de San Carlos de Guatemala. El objetivo del proyecto es modelar el comportamiento de un agente inteligente que opera en un entorno hostil y dinámico, con la misión de salvar personas atrapadas en un área peligrosa. Este entorno, simulado en una plataforma web tridimensional, se construye a partir de configuraciones definidas en un archivo JSON, que incluye la estructura del mapa, zonas peligrosas, obstáculos, ubicación de las personas a rescatar, y parámetros de comportamiento del entorno.

El robot simulado debe recorrer el mapa desde una zona segura y rescatar a las personas en un orden preestablecido, llevándolas una por una de vuelta a la zona segura, mientras el entorno se vuelve cada vez más inestable. A lo largo de la simulación, nuevas zonas peligrosas pueden emerger de forma aleatoria o contigua, según una probabilidad configurable por el usuario. Esto exige que el sistema reaccione en tiempo real, replantee rutas, evite caminos bloqueados y tome decisiones inteligentes si un objetivo ya no es alcanzable.

El proyecto también requiere la implementación de tres algoritmos de búsqueda distintos, cuya eficiencia será comparada mediante simulaciones repetidas. Los estudiantes deberán evaluar el rendimiento de cada algoritmo según el número de personas rescatadas y el tiempo de ejecución, promoviendo así la reflexión crítica sobre la aplicabilidad de distintas técnicas de IA en entornos cambiantes.

PathGuard representa un ejercicio integral de aplicación de conocimientos de simulación, lógica, estructuras de datos y algoritmos de búsqueda, al mismo tiempo que introduce elementos de razonamiento adaptativo, interacción con archivos estructurados, y visualización en tiempo real. Es un proyecto orientado a fomentar habilidades técnicas avanzadas, pensamiento lógico y la toma de decisiones en escenarios de alta incertidumbre.

2. Competencias que desarrollaremos

- Integra fundamentos de lógica, realidad aumentada y simulación robótica mediante el uso de entornos de desarrollo como Tau Prolog, AR.js y Three.js para resolver problemas relacionados a inferencia, interacción digital y simulación de comportamientos autónomo.
- Implementa soluciones de inteligencia artificial empleando técnicas de inferencia lógica, diseño de entornos virtuales y control de robots en distintos escenarios de optimización, clasificación y toma de decisiones.
- Integra diseños 2D y 3D de robots utilizando librerías como p5.js y Three.js en simulaciones programadas que involucren acciones y comportamientos dinámicos.

3. Objetivos del Aprendizaje

3.1 Objetivo General

Al finalizar el proyecto, los estudiantes serán capaces de diseñar e implementar un sistema de simulación 3D que represente un entorno de rescate autónomo en condiciones dinámicas, integrando algoritmos de búsqueda, estructuras de datos, visualización en tres dimensiones y lógica adaptativa. El sistema será capaz de interpretar configuraciones definidas en archivos JSON y adaptarse en tiempo real a condiciones cambiantes, aplicando los fundamentos de la Inteligencia Artificial vistos en el curso.

3.2 Objetivos Específicos

Al finalizar el proyecto, los estudiantes deberán ser capaces de:

1. **Construir una simulación funcional y dinámica**, que represente un entorno tridimensional con obstáculos, zonas peligrosas y personas a rescatar, donde el robot actúe de forma autónoma guiado por configuraciones definidas en un archivo JSON editable.
2. **Implementar y analizar comparativamente tres algoritmos de búsqueda**, midiendo su desempeño bajo condiciones cambiantes del entorno y documentando su eficacia en términos de personas rescatadas, tiempo de ejecución y capacidad de adaptación.
3. **Diseñar una lógica de actualización en tiempo real del entorno**, donde nuevas zonas peligrosas aparezcan de forma probabilística, obligando al robot a replantear rutas, omitir objetivos inalcanzables, priorizar la seguridad y concluir la simulación con un resumen detallado de resultados.

4. Enunciado del Proyecto

4.1 Descripción del problema a resolver

En escenarios de emergencia como terremotos, incendios o derrumbes, uno de los retos más urgentes es la localización y rescate de personas atrapadas dentro de estructuras colapsadas o áreas peligrosas. Coordinar estas operaciones requiere inteligencia, adaptabilidad y eficiencia, especialmente cuando el entorno es inestable y las condiciones pueden cambiar de forma repentina. Simular este tipo de entornos permite comprender mejor los desafíos técnicos y estratégicos involucrados en operaciones de rescate autónomas.

Este proyecto plantea el desarrollo de PathGuard, un simulador tridimensional de rescate inteligente, en el que un robot autónomo se desplaza por un entorno hostil para localizar y

evacuar a personas atrapadas, priorizando su seguridad y adaptándose a la evolución del terreno. La aplicación utilizará un entorno 3D desarrollado con Three.js y p5.js, y permitirá la carga dinámica de escenarios a través de archivos JSON estructurados. Estos escenarios incluirán: el plano general del entorno, la ubicación del robot inicial, los puntos donde se encuentran las personas por rescatar, zonas peligrosas y obstáculos.

El objetivo del robot es rescatar a las personas una por una, en el orden indicado en el archivo JSON. Deberá llevarlas a la zona segura y luego continuar con la siguiente. Durante la simulación, nuevas zonas peligrosas surgirán dinámicamente con una probabilidad configurada por el usuario. Estas zonas podrán expandirse desde zonas peligrosas existentes o emerger en ubicaciones aisladas, dependiendo del parámetro de probabilidad definido. Cada cierto número de segundos se evaluará si se agrega una nueva zona peligrosa y en qué lugar, modificando así las rutas disponibles en tiempo real.

Si durante la ejecución alguna ruta se vuelve inaccesible por la aparición de nuevas zonas peligrosas, el robot deberá replanificar una nueva ruta hacia la siguiente persona en la lista (respetando el orden). Si no existen rutas viables hacia ningún objetivo restante, el robot deberá regresar a la zona segura, si esta sigue accesible. En caso de quedar atrapado o sin salida, la simulación finalizará mostrando un mensaje de Game Over e incluyendo al robot en el conteo de personas no rescatadas.

Al final de la simulación, se deberá mostrar un informe con el número total de personas rescatadas y cuántas no pudieron ser alcanzadas. El sistema también debe permitir configurar el intervalo de aparición de nuevas zonas peligrosas (en segundos), así como el porcentaje de probabilidad de que estas se generen de forma contigua a zonas existentes o como focos aislados. Todo esto permite un entorno altamente dinámico, que obliga a la replanificación constante y pone a prueba la eficiencia de los algoritmos implementados.

El propósito de este proyecto es que los estudiantes comprendan los principios de la inteligencia artificial aplicada a la planificación en tiempo real, la simulación de agentes autónomos, la eficiencia algorítmica en entornos cambiantes y el diseño de sistemas adaptativos.

4.2 Alcance del proyecto

El sistema PathGuard deberá ser implementado como una aplicación web tridimensional, accesible desde navegadores modernos sin necesidad de instalación. Su estructura estará basada en HTML, CSS y JavaScript puro, con apoyo de las bibliotecas p5.js y Three.js para la representación visual y la animación del entorno.

El entorno tridimensional debe incluir los controles de órbita respectivos para poder visualizar todo el espacio de forma fácil y detallada.

Al inicio del proyecto, el usuario deberá poder cargar un archivo JSON que defina completamente el escenario de simulación, incluyendo:

- Dimensiones del laberinto (alto x ancho).
- Posición inicial del robot.
- Coordenadas de las personas a rescatar (en el orden en que deben ser salvadas).

- Coordenadas de la zona segura (área de evacuación).
- Obstáculos y zonas peligrosas iniciales.

Ejemplo:

```
{
  "ancho": 7,
  "alto": 7,
  "inicio": [0, 0],
  "peligros": [[2, 2], [4, 4]],
  "muros": [[1, 1], [1, 2], [3, 3]],
  "personas": [[6, 0], [2, 5], [5, 6]]
}
```

El entorno será renderizado como una cuadrícula 3D, con diferentes texturas y modelos que distingan zonas seguras, peligrosas, obstáculos, personas atrapadas y caminos transitables. El robot será representado mediante un modelo animado que se desplazará paso a paso por el entorno.

El usuario deberá poder seleccionar uno de los tres algoritmos de búsqueda implementados (por ejemplo: BFS, A*, Dijkstra), y la simulación comenzará automáticamente con la ruta hacia el primer objetivo. El robot ejecutará la ruta paso a paso según se realiza la evaluación del algoritmo, no solamente la ruta final. Su desplazamiento debe ser animado, recogiendo al objetivo, y llevándolo de vuelta a la zona segura. Una vez entregado, comenzará la búsqueda del siguiente.

Mientras la simulación esté en curso, el sistema evaluará periódicamente (cada X segundos) si se deben generar nuevas zonas peligrosas. El valor de X, así como la probabilidad de que las nuevas zonas se formen contiguamente a zonas existentes o de forma aislada, serán configurables antes de iniciar la simulación. Esta variabilidad obligará al sistema a recalcular rutas dinámicamente si las condiciones cambian. Si durante la operación una ruta hacia el objetivo actual se vuelve inaccesible, el robot debe pasar al siguiente objetivo disponible. Solo si no hay más objetivos rescatables, deberá regresar a la zona segura.

La simulación terminará cuando:

- Todas las personas han sido rescatadas
- No existen rutas viables hacia los objetivos restantes y el robot regresa a la zona segura
- El robot queda completamente encerrado, sin opción de rescate o retorno.

El sistema debe mostrar un reporte final al usuario con:

- Número de personas rescatadas.
- Número de personas no alcanzadas.
- Algoritmo utilizado.
- Tiempo de simulación.
- Ruta final del robot.

Adicionalmente, se deberá entregar un documento técnico que explique los tres algoritmos implementados, su funcionamiento, su aplicabilidad en el contexto del proyecto y un análisis comparativo con al menos tres ejecuciones por algoritmo. Este análisis debe incluir métricas como tiempo de ejecución, eficiencia de rutas y número de personas salvadas.

4.3 Requerimientos técnicos

Para el desarrollo del proyecto PathGuard, los estudiantes deberán emplear tecnologías del lado del cliente, sin el uso de servidores backend, frameworks modernos ni bases de datos externas complejas. La intención es que los estudiantes comprendan profundamente los fundamentos de la simulación de entornos 3D, la inteligencia artificial simbólica y el control de agentes autónomos a través de algoritmos de búsqueda aplicados en tiempo real.

Las herramientas y tecnologías requeridas son:

- **Lenguajes de programación:**
El sistema debe ser desarrollado utilizando únicamente HTML, CSS y JavaScript puro, sin frameworks modernos (como React, Angular o Vue) ni runtimes (como Node.js o Deno). Esta limitación busca fortalecer la lógica de programación, la manipulación de estructuras de datos, y el diseño desde cero de soluciones funcionales e interactivas.
- **Bibliotecas Gráficas:**
El entorno tridimensional del simulador debe construirse utilizando Three.js para la renderización 3D, y p5.js para efectos visuales adicionales o interfaces en 2D si se requieren. Además se deben incluir los controles de órbita de Three.js.
- **Algoritmos de Búsqueda:**
Los estudiantes deberán implementar **tres algoritmos diferentes de búsqueda** (por ejemplo, BFS, DFS, A*, Dijkstra o Greedy), y estos deben poder ejecutarse bajo demanda durante la simulación. La elección y funcionamiento de cada algoritmo deberá justificarse en la documentación técnica, además de incluir un comparativo de desempeño entre ellos con base en al menos tres simulaciones reales. Los resultados del comparativo deberán considerar el tiempo de ejecución, el número de personas rescatadas y la cantidad de movimientos realizados por el robot.
- **Archivos JSON de configuración:**
Toda la estructura del escenario será definida mediante un archivo JSON que debe poder ser cargado desde la interfaz. Este archivo incluirá:
 - Dimensiones del entorno.
 - Ubicación del punto de inicio (zona segura).
 - Posición de obstáculos.
 - Ubicación inicial de zonas peligrosas.
 - Lista ordenada de personas a rescatar (con coordenadas específicas).
- **Parámetros configurables:**
Antes de iniciar cada simulación, el sistema debe permitir al usuario configurar los siguientes elementos:

- Algoritmo de búsqueda a utilizar.
 - Tiempo (en segundos) entre cada generación de nuevas zonas peligrosas.
 - Probabilidad de expansión contigua vs. expansión aleatoria de nuevas zonas peligrosas. Este valor debe ser ajustable mediante un control deslizante, de forma que el usuario pueda, por ejemplo, establecer un 60% de aparición contigua y 40% no contigua.
- **Mecanismo de evaluación en tiempo real:**
Durante la simulación, el robot deberá adaptarse a los cambios en el entorno. Si la ruta actual hacia una persona se bloquea por una zona peligrosa que apareció, el robot deberá interrumpir su trayecto y recalcular una nueva ruta hacia la siguiente persona en el orden definido por el archivo JSON. Solo en caso de que no haya personas restantes por rescatar o todas estén inalcanzables, el robot deberá retornar a la zona segura. Si el acceso a la zona segura también es bloqueado, se considerará "Game Over" y el robot será contabilizado como una víctima más en el informe final.
 - **Control de versiones y publicación:**
Todo el proyecto deberá estar alojado en un repositorio privado en GitHub durante su desarrollo, cuyo nombre será: Pendiente, agregar la carpeta "Proyecto3".

El auxiliar robinbuezo11 deberá ser agregado como colaborador con permisos completos. Una vez calificado, el repositorio deberá hacerse público. La aplicación final deberá ser publicada mediante **GitHub Pages**.
 - **Licencia:**
El proyecto debe ser **Open Source con licencia MIT**, la cual debe incluirse explícitamente en el repositorio.

4.4 Entregables

Tipo	Descripción
Aplicación Web	Enlace funcional a la aplicación web publicada mediante GitHub Pages . Debe estar alojada dentro del mismo repositorio del proyecto.
Manual Técnico	Documento PDF que incluya la arquitectura del sistema, flujos de interacción, explicación detallada de los algoritmos implementados y su lógica de rescate. Debe explicar también cómo se diseñó el mecanismo de generación dinámica de zonas peligrosas y cómo se gestiona la replanificación en tiempo real.
Manual de Usuario	Documento PDF destinado al usuario final. Debe contener una guía paso a paso sobre el uso de la interfaz, carga de archivos, parámetros configurables, control del simulador y lectura de resultados finales. Debe incluir capturas de pantalla e instrucciones claras.

Comparativo de Algoritmos	Documento adicional que explique el funcionamiento de cada uno de los algoritmos elegidos y además contenga al menos tres ejecuciones por cada uno de los algoritmos de búsqueda implementados, describiendo los resultados en cuanto a número de personas rescatadas, tiempo de ejecución y movimientos realizados. El documento debe concluir cuál fue el algoritmo más eficiente y por qué.
Código Fuente	Repositorio de GitHub con el código organizado, comentado y funcional. Debe contener los archivos HTML, CSS, JavaScript, archivos JSON de prueba, documentación, imágenes y licencia MIT. Debe reflejar buenas prácticas de desarrollo y estar versionado correctamente.

- **Fecha límite de entrega:** Pendiente.
- **Colaborador obligatorio en el repositorio:** El auxiliar *robinbuezo11* debe ser agregado al repositorio como colaborador con permisos de lectura completa.
- **Nombre del repositorio:** Pendiente. Agregar la carpeta llamada "Proyecto3"
- **Medio de entrega:** El enlace al repositorio y al sitio web publicado debe ser enviado a través de la plataforma UEDI

5. Metodología

El desarrollo del proyecto PathGuard deberá seguir una metodología progresiva y estructurada, que permita a los estudiantes construir una solución robusta, funcional y bien documentada. Dado el enfoque técnico del proyecto y la integración de lógica autónoma con entornos tridimensionales dinámicos, se recomienda trabajar bajo principios de desarrollo ágil, utilizando metodologías como SCRUM para organizar el trabajo por sprints, realizar entregas iterativas, y fomentar la colaboración continua en equipo.

A continuación, se describen las fases mínimas que deberán cubrirse para completar satisfactoriamente el proyecto:

1. Investigación preliminar

La primera etapa consistirá en una revisión conceptual y técnica de los algoritmos de búsqueda más relevantes en entornos de navegación (como BFS, DFS, A*, Dijkstra), y su aplicabilidad dentro de entornos con cambios dinámicos. Además, los estudiantes deberán estudiar las funcionalidades de las bibliotecas Three.js y p5.js, especialmente su integración para representar escenarios tridimensionales y simular movimiento y animación de objetos en tiempo real. También se investigará sobre técnicas de replanificación dinámica, gestión de estructuras JSON y diseño de agentes autónomos con lógica condicional adaptable.

2. Diseño conceptual y de simulación

En esta fase se diseñará la estructura general del sistema. Los estudiantes deberán crear diagramas de flujo del comportamiento del agente (robot), bocetos de la interfaz visual 3D, y definir la estructura inicial del archivo JSON. Este archivo incluirá información clave como dimensiones del entorno, ubicación inicial del robot,

obstáculos, zonas peligrosas iniciales, personas a rescatar y parámetros configurables como el intervalo de aparición de nuevas amenazas y la probabilidad de expansión contigua o aleatoria. También se definirá cómo se calculará el "Game Over", el criterio de éxito o fracaso del rescate, y la visualización de estadísticas finales.

3. Construcción del entorno tridimensional

Utilizando Three.js, se deberá construir el mapa del simulador, representando visualmente los muros, caminos libres, personas atrapadas, zonas peligrosas y la zona segura. Se desarrollará también un modelo del robot que se moverá de forma autónoma dentro del entorno. Este modelo puede utilizar una forma geométrica simple o un diseño personalizado, y debe incluir una animación clara de movimiento. Además, se deberá implementar una cámara con control de perspectiva y zoom que permita observar el entorno con comodidad durante la simulación.

4. Integración del archivo JSON y configuración dinámica

Se desarrollará el mecanismo que permitirá cargar desde la interfaz un archivo JSON de configuración del entorno. Este archivo deberá ser leído en tiempo real y utilizado para renderizar el escenario. A su vez, se deberá permitir que los valores del archivo se puedan configurar antes de iniciar la simulación: se podrá seleccionar el algoritmo de búsqueda, definir el tiempo entre eventos dinámicos (nuevas zonas peligrosas) y establecer la probabilidad de expansión contigua (porcentaje ajustable). También deberá permitirse la exportación del archivo JSON con las estadísticas al finalizar la simulación, incluyendo cuántas personas fueron rescatadas, cuántas no se pudieron salvar y cuántas zonas se volvieron intransitables.

5. Implementación del sistema de rescate y algoritmo de replanificación

Se deberán implementar tres algoritmos de búsqueda seleccionados por los estudiantes, integrados a un sistema de navegación autónoma que tome decisiones en función del estado actual del entorno. El robot siempre deberá intentar rescatar a las personas en el orden especificado en el archivo JSON, y al completar cada rescate, deberá regresar a la zona segura antes de proceder al siguiente. En caso de que una zona peligrosa bloquee el acceso al objetivo actual, el robot deberá intentar alcanzar al siguiente objetivo disponible sin regresar a la base. Solo cuando no existan personas por rescatar o todas sean inalcanzables, el agente deberá retornar a la zona segura, y si este retorno se bloquea, deberá marcarse como "no rescatado".

La aparición de nuevas zonas peligrosas deberá ejecutarse cada cierta cantidad de segundos y seguir las probabilidades definidas por el usuario para generar dichas zonas de forma contigua o aleatoria.

6. Pruebas funcionales y comparativo de algoritmos

Se realizarán pruebas funcionales del sistema con distintos laberintos JSON, evaluando tanto la ejecución como la capacidad de adaptación del robot ante cambios dinámicos. Además, se deberá ejecutar el simulador al menos tres veces con cada uno de los algoritmos implementados, registrar los resultados y elaborar un comparativo en el cual se analicen la eficiencia, personas rescatadas, rutas recorridas

y tiempo total de ejecución. Este análisis será parte del entregable de evaluación técnica.

7. Documentación técnica y de usuario

Una vez completado el desarrollo, los estudiantes deberán generar dos documentos obligatorios. El primero será el manual técnico, donde se explique detalladamente la arquitectura del sistema, las decisiones de diseño, la lógica de los algoritmos y la estructura del archivo JSON. El segundo será el manual de usuario, con instrucciones claras para operar la simulación, modificar parámetros, cargar archivos y comprender los resultados. Ambos documentos deben estar acompañados de capturas y esquemas explicativos.

8. Presentación final del proyecto

Finalmente, cada grupo presentará su proyecto, demostrando el funcionamiento del simulador en vivo, explicando su lógica de rescate, su integración dinámica y sus decisiones técnicas. Durante esta presentación se evaluará el dominio del contenido, la participación grupal y la claridad en la exposición.

6. Desarrollo de Habilidades Blandas

Para complementar las habilidades técnicas desarrolladas durante el diseño e implementación de PathGuard, los estudiantes deberán fortalecer un conjunto de competencias blandas que son indispensables para desenvolverse en ambientes profesionales multidisciplinarios. Este proyecto ha sido concebido no solo como una oportunidad para aplicar conocimientos técnicos en programación, simulación y lógica de búsqueda, sino también como un ejercicio de colaboración, liderazgo, organización y comunicación.

6.1 Proyectos en Grupo

El proyecto está diseñado para ser realizado de forma colaborativa en equipos de trabajo, permitiendo que cada miembro explore y potencie sus capacidades individuales dentro de un entorno grupal. La división de roles, la asignación de tareas, el control del tiempo y la coordinación constante serán elementos fundamentales para lograr el éxito del proyecto.

Cada grupo deberá establecer claramente sus funciones internas desde el inicio del proyecto. Se recomienda asignar responsables para los distintos componentes clave: diseño del entorno 3D, implementación de algoritmos de búsqueda, control de eventos dinámicos, integración del archivo JSON, pruebas y documentación. No obstante, todos los miembros deberán involucrarse en las decisiones colectivas y comprender el funcionamiento integral del sistema.

Para coordinar el trabajo de forma eficiente, se sugiere el uso de herramientas colaborativas como Trello, Notion o ClickUp para la gestión de tareas; Google Docs o Overleaf para la redacción compartida de documentación; y GitHub como sistema de control de versiones y

plataforma de colaboración técnica. Estas herramientas permitirán mantener un flujo de trabajo ordenado, transparente y con historial de avances fácilmente verificable.

6.1.1 Trabajo en Equipo

Los estudiantes deberán desarrollar la capacidad de organizarse como equipo técnico, tomando decisiones conjuntas, distribuyendo la carga de trabajo de forma equitativa y apoyándose mutuamente en la resolución de obstáculos. La experiencia de trabajo colaborativo contribuirá al desarrollo de habilidades como el liderazgo, la empatía profesional, la toma de decisiones colectiva y la ejecución disciplinada bajo presión.

6.1.2 Comunicación Efectiva

La comunicación dentro del equipo deberá ser clara, frecuente y orientada a la resolución de objetivos. Se fomentará la realización de reuniones periódicas para revisar avances, discutir bloqueos técnicos y proponer soluciones. Además, cada equipo deberá mantener contacto activo con el auxiliar del curso, quien ofrecerá orientación y retroalimentación en cada fase del desarrollo. La documentación continua de decisiones y cambios será una práctica obligatoria.

6.1.3 Resolución de Conflictos

Durante el proceso de desarrollo pueden surgir diferencias en criterios técnicos, prioridades de implementación o asignación de tareas. Los equipos deberán abordar estos desafíos con herramientas de diálogo, análisis colectivo y enfoque en objetivos comunes. Se espera que los estudiantes muestren disposición a escuchar, negociar y ajustar sus decisiones en beneficio del rendimiento general del proyecto.

6.2 Proyectos Individuales

En casos autorizados por el auxiliar, un estudiante podrá asumir el desarrollo de *PathGuard* de manera individual. Esta modalidad exigirá una mayor autodisciplina, responsabilidad y capacidad de organización.

6.2.1 Autogestión del Tiempo

El estudiante deberá planificar su trabajo de forma autónoma, estableciendo metas semanales claras, monitoreando su avance y ajustando sus tiempos según las exigencias técnicas del proyecto. El uso de cronogramas personales, listas de tareas o técnicas de productividad será fundamental para alcanzar los entregables en el plazo previsto.

6.2.2 Responsabilidad y Compromiso

El estudiante asumirá el control completo de todas las fases del proyecto, desde la concepción del entorno 3D, la lógica del agente de rescate, la configuración de eventos dinámicos, hasta las pruebas y la documentación. Esta modalidad reforzará su sentido de compromiso, permitiéndole desarrollar un perfil técnico integral y enfrentar desafíos sin depender del soporte directo de otros compañeros.

6.2.3 Resolución de Problemas

El trabajo individual impulsará al estudiante a resolver problemas por cuenta propia, investigar soluciones técnicas, depurar errores y diseñar estrategias de adaptación. Esta experiencia fortalecerá su capacidad de análisis lógico, pensamiento crítico y autonomía técnica.

6.2.4 Reflexión Personal

Una vez finalizado el proyecto, el estudiante deberá realizar una autoevaluación reflexiva sobre su proceso de trabajo, las herramientas utilizadas, las decisiones tomadas, los resultados obtenidos y las posibles mejoras para futuras versiones. Este ejercicio fomentará la capacidad de análisis postmortem, esencial para el aprendizaje continuo y la mejora profesional.

7. Cronograma

El desarrollo del proyecto PathGuard se organizará en dos entregas clave. La primera entrega será evaluada como una tarea parcial (Tarea 3) para asegurar que los equipos han iniciado el desarrollo de forma adecuada, mientras que la segunda corresponderá a la entrega final completa (Proyecto 3). Cada entrega tendrá plazos específicos y será evaluada de forma independiente.

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto		
Entrega No. 1 – Tarea 3		
Entrega No. 2 – Proyecto Final		
Calificación		

Detalle por entrega

- Tarea 3 (Entrega No. 1)**
 Deberá incluir la estructura básica del simulador, la lectura del archivo JSON con al menos un ejemplo de entorno funcional, una interfaz inicial del entorno 3D renderizado, y la implementación de al menos un algoritmo de búsqueda que permita simular un rescate básico. Además, se deberá presentar un avance del archivo de documentación técnica y el repositorio GitHub debe estar activo.
- Proyecto Final (Entrega No. 2)**
 Debe entregarse el proyecto completo, funcional e integrado con las zonas peligrosas dinámicas, rescate ordenado de objetivos, replanificación, control de probabilidades de aparición, reporte de resultados y los tres algoritmos de búsqueda comparados. La documentación técnica, el manual de usuario y el análisis comparativo de los algoritmos deben estar finalizados y entregados en el mismo repositorio.

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de ser evaluado, el proyecto deberá cumplir con los siguientes requisitos mínimos. Si alguno de estos no se cumple, el proyecto no podrá ser calificado y se considerará como no entregado o incompleto:

Tema	Descripción	Cumple (Sí/No)
Cumplimiento de la tecnología establecida	El sistema debe estar desarrollado con HTML, CSS, JS puro, p5.js y Three.js. No se permite el uso de frameworks ni runtimes externos.	
Gestión y entregas del proyecto	El proyecto debe entregarse en dos fases, con control de versiones en GitHub. Se debe agregar al usuario <i>robinbuezo11</i> como colaborador.	
Documentación obligatoria	Manual técnico, manual de usuario y análisis comparativo deben estar incluidos.	
Funcionalidad mínima	El simulador debe ejecutarse y cumplir con la lógica base del rescate.	

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Conocimiento	90	
2. Habilidades	10	
Documentación	4	
Preguntas	6	
TOTAL	100	

8.3 Detalle de la Calificación

Descripción de Ponderación	Valor	Observación	Punteo
1. Conocimiento	90		
Lectura de archivo JSON	5	El sistema lee y construye entornos con base en archivos JSON correctamente estructurados.	

Renderización del entorno 3D	5	El escenario es visualmente comprensible y correctamente renderizado con Three.js.	
Movimiento del Robot	5	El robot se mueve de forma coherente, animada y en tiempo real.	
Rescatar en orden	5	El robot salva personas respetando el orden del JSON.	
Algoritmos de búsqueda	15	Implementación funcional de tres algoritmos distintos.	
Dinámica de zonas peligrosas	5	Aparición de zonas peligrosas con probabilidades configurables.	
Replanificación de ruta	10	El robot es capaz de recalcular ruta si un objetivo queda bloqueado.	
Retorno a zona segura y finalización lógica	5	El simulador finaliza correctamente informando personas rescatadas y no rescatadas.	
Análisis comparativo	5	Documento que compara los algoritmos en función de tiempo y efectividad.	
Ejecución paso a paso visual de cada algoritmo	30	La ejecución del algoritmo se visualiza paso a paso en base a la lógica de búsqueda.	
2. Habilidades	10		
Documentación	4		
Manual Técnico	2	Estructura, explicación del JSON, algoritmos utilizados y decisiones implementadas.	
Manual de Usuario	2	Guía clara con capturas de pantalla para la ejecución y uso del simulador.	
Preguntas	6		
Pregunta 1	2	Pregunta teórica o sobre el desarrollo del proyecto	
Pregunta 2	2	Pregunta teórica o sobre el desarrollo del proyecto	
Pregunta 3	2	Pregunta teórica o sobre el desarrollo del proyecto	
Penalizaciones			
Entrega fuera del plazo establecido	-20	Solo si se entrega fuera del plazo oficial (En caso de que la calificación sea mucho tiempo después de la entrega la penalización puede ser mayor).	

Impuntualidad en la calificación	-20	En caso de impuntualidad en la calificación.	
TOTAL	100		

8.4 Valores

Durante el desarrollo del proyecto PathGuard, se espera que los estudiantes actúen con integridad académica, responsabilidad técnica y ética profesional. Todos los entregables deben ser producto del trabajo original del equipo o estudiante individual, sin recurrir a copias no autorizadas ni reutilización indebida de materiales ajenos.

1. Originalidad del Trabajo

Cada estudiante o equipo debe desarrollar su propio código, simulador, lógica de comportamiento, archivo de configuración (JSON), algoritmos de búsqueda y documentación. Se espera que el resultado sea completamente original, basado en el aprendizaje adquirido durante el curso. Todo el trabajo entregado debe reflejar comprensión técnica y participación de los autores.

2. Prohibición de Copias y Plagio

Queda estrictamente prohibido copiar, replicar o adaptar total o parcialmente el código, documentación o cualquier componente del proyecto desde otras fuentes sin el debido análisis, modificación y referencia.

- La detección de plagio (entre compañeros, de internet o de ciclos anteriores) será penalizada con calificación de 0 puntos.
- Los responsables serán **reportados formalmente a la Escuela de Ciencias y Sistemas**.
- Esto incluye el uso de ediciones superficiales de código ajeno, sin comprensión real ni justificación lógica.

3. Uso Responsable de Recursos Externos

Está permitido el uso de bibliotecas de consulta, ejemplos o fragmentos educativos siempre y cuando:

- Se referencien adecuadamente en el código o en los anexos.
- Se comprenda completamente su funcionamiento.
- No se utilicen como sustituto de la lógica propia del proyecto. Cualquier duda deberá ser consultada con el catedrático o auxiliar antes de su uso.

4. Revisión y Validación del Trabajo

El equipo docente podrá utilizar herramientas automáticas y revisiones manuales para comparar entregas y detectar similitudes no justificadas.

- En caso de sospecha, el estudiante deberá defender su solución, explicar sus decisiones y justificar el funcionamiento de su código y todo lo relacionado.
- Si no logra demostrar la autoría o comprensión del trabajo, se asignará **una calificación de 0 puntos**, sin opción a apelación académica.

5. Licencia y Transparencia del Proyecto

- El proyecto debe ser **OpenSource** bajo licencia **MIT**.
- Todo el código, marcadores, documentación técnica, pruebas y manuales deberán estar alojados en un repositorio de GitHub privado durante el desarrollo.
- Una vez calificado, el repositorio deberá hacerse público.
- El auxiliar del curso (**robinbuezo11**) debe ser agregado como colaborador en el repositorio.
- El nombre del repositorio debe seguir el formato: Pendiente. Agregar la carpeta llamada "Proyecto3"

6. Entrega y Fechas Límite

- No se permitirá realizar modificaciones al código ni a la documentación después de la fecha de entrega final establecida en el cronograma.
- Las entregas fuera del plazo establecido pueden considerarse no válidas y se aplicarán las penalizaciones indicadas, a menos que se haya aprobado una prórroga justificada con anterioridad.